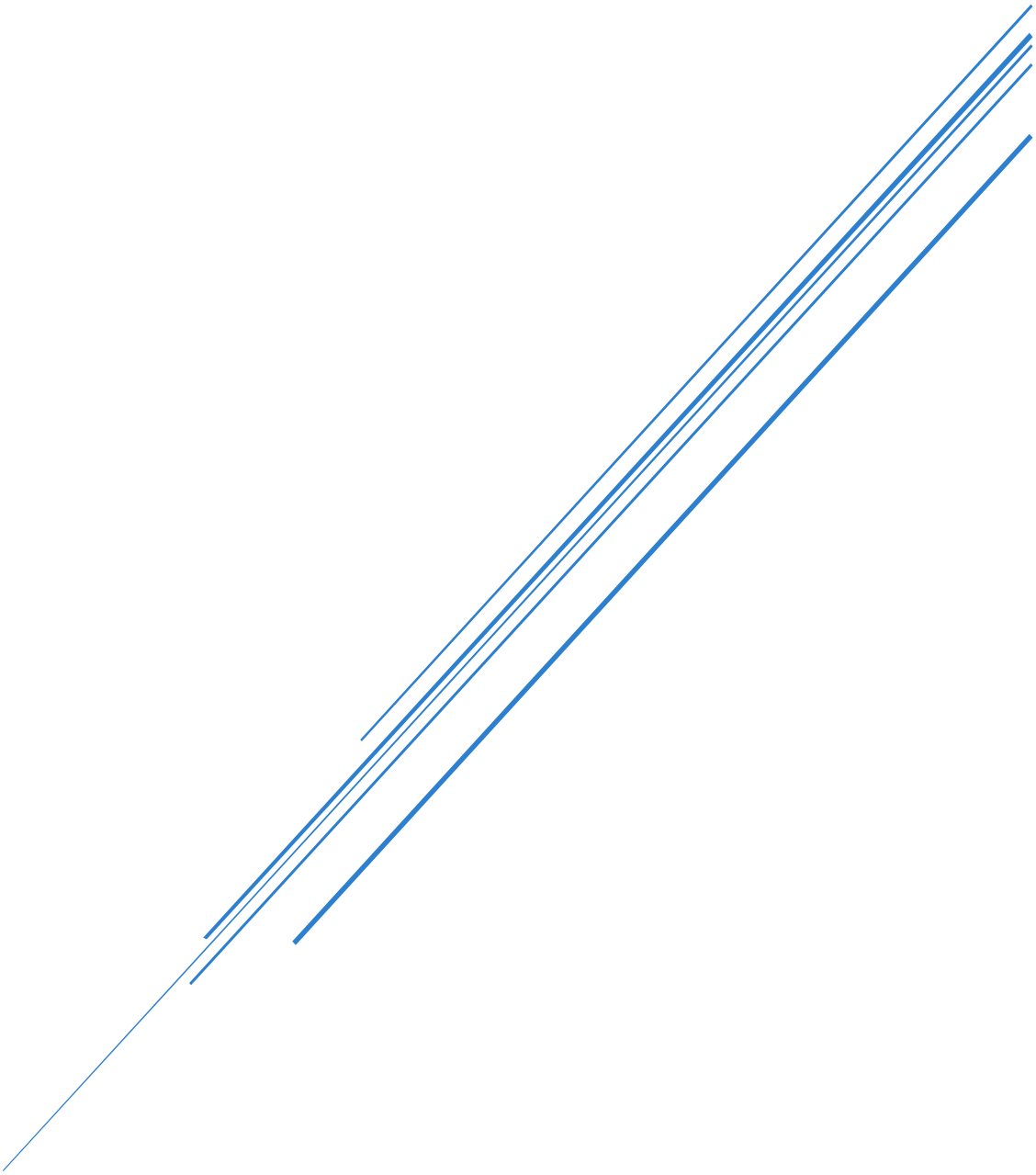
COMPTE RENDU TP 2 - INFRASTRUCTURE DE GESTION DE CLÉS

Étudiant : CISSE Abdoul Wahabou



ECOLE POLYTECHNIQUE DE OUAGADOUGOU

Module EC2 - Ingénierie de la cryptographie -Dr Zakaria Sawadogo

Introduction

Dans ce TP, j'ai construit avec OpenSSL une petite infrastructure de gestion de clés (PKI) : une autorité de certification (AC) racine, une AC intermédiaire, puis un certificat de serveur et un certificat de client. J'ai ensuite validé les chaînes de certificats, révoqué un certificat et automatisé la construction dans un script (pki.sh).

Élément	Clé et durée	Rôle
AC racine	P-384, 10 ans, hors ligne	Sommet de la confiance. Signe uniquement l'intermédiaire et ses listes de révocation
AC intermédiaire	P-256, 5 ans	Signe les certificats du quotidien (serveur, client). Limitée à tp.local
Serveur www.tp.local	P-256, 90 jours	Prouve l'identité du serveur (usage « serveur TLS »)
Client alice@tp.local	P-256, 90 jours	Prouve l'identité de l'utilisateur (usage « client TLS »)

Les certificats servent à lier une clé publique à une identité. Une AC de confiance signe ce lien. On garde la racine hors ligne et on utilise l'intermédiaire au quotidien : si l'intermédiaire est compromis, la racine reste intacte.

1. Rôle des lignes des sections d'extensions

Le mot critical oblige un logiciel qui ne comprend pas l'extension à refuser le certificat. Le tableau explique chaque ligne des sections ext_inter, ext_serveur et ext_client.

Ligne	Section	Rôle
basicConstraints = critical, CA:TRUE, pathlen:0	ext_inter	Ce certificat est une AC. pathlen:0 : elle ne peut signer que des certificats de fin de chaîne, pas d'autres AC (la racine, elle, a pathlen:1)
keyUsage = critical, keyCertSign, cRLSign	ext_inter	La clé sert uniquement à signer des certificats et des listes de révocation
nameConstraints = critical, permitted;DNS:tp.local, permitted;email:tp.local	ext_inter	L'intermédiaire ne peut certifier que des noms de domaine et des adresses e-mail en tp.local
basicConstraints = critical, CA:FALSE	ext_serveur, ext_client	Certificat de fin de chaîne : il ne peut signer aucun autre certificat
keyUsage = critical, digitalSignature	ext_serveur, ext_client	La clé sert à signer, ce qui prouve l'identité pendant la poignée de main TLS
extendedKeyUsage = serverAuth (ou clientAuth)	ext_serveur (ext_client)	Usage autorisé : serveur TLS ou client TLS, jamais les deux
subjectKeyIdentifier = hash	toutes	Empreinte de la clé du certificat
authorityKeyIdentifier = keyid:always	ext_inter, ext_serveur, ext_client	Empreinte de la clé de l'émetteur : relie chaque certificat à celui qui l'a signé
subjectAltName (dans la demande)	CSR serveur et client	Noms pour lesquels le certificat est valable (www.tp.local, tp.local ; alice@tp.local), recopiés par copy_extensions = copy

2. Réalisation et observations

AC racine (étape 2)

La clé est une clé à courbe elliptique P-384 et le certificat est signé avec ecdsa-with-SHA384. Il est valable 10 ans (du 25 octobre 2026 au 22 octobre 2036). Les extensions critiques sont Basic Constraints (CA:TRUE, pathlen:1) et Key Usage (Certificate Sign, CRL Sign). L'émetteur et le sujet sont identiques (C = BF, O = Universite TP, CN = Racine TP) : la racine est au sommet de la hiérarchie, aucune autorité n'est au-dessus d'elle pour la signer, elle signe donc son propre certificat. Cette signature ne prouve rien par elle-même : on lui fait confiance parce qu'elle est installée d'avance comme point d'ancrage de confiance.

AC intermédiaire (étape 3)

L'intermédiaire a créé sa propre clé (P-256) et une demande de certificat (CSR), puis la racine l'a signée pour 5 ans. La vérification donne OK. Le certificat contient bien CA:TRUE, pathlen:0 et les contraintes de nom permises DNS:tp.local et email:tp.local.

Certificats de serveur et de client (étape 4)

Le serveur www.tp.local et le client alice@tp.local ont chacun généré leur clé et leur demande ; l'intermédiaire les a signées pour 90 jours. Les deux ont CA:FALSE et Digital Signature. Le serveur a l'usage TLS Web Server Authentication et les noms alternatifs www.tp.local et tp.local. Le client a l'usage TLS Web Client Authentication et l'adresse alice@tp.local dans le SAN.

Validations (étape 5)

Commande (openssl verify)	Résultat	Interprétation
-verify_hostname www.tp.local serveur.crt	OK	Chaîne valide et nom présent dans le SAN
-verify_hostname autre.example serveur.crt	erreur 62 : hostname mismatch	Même certificat, mais il n'a pas été émis pour ce nom
-purpose sslclient alice.crt	OK	Alice peut servir de client TLS
example.crt (www.example.com)	erreur 47 : permitted subtree violation	Nom hors de tp.local : refusé par celui qui valide la chaîne

L'AC a pu signer le certificat de www.example.com parce que la signature (openssl ca) ne contrôle pas les contraintes de nom. Ces contraintes sont écrites dans le certificat de l'intermédiaire par la racine, et c'est celui qui valide la chaîne (navigateur, serveur, openssl verify) qui les applique. Elles limitent donc les dégâts si l'intermédiaire est compromis.

Révocation (étape 6)

J'ai supposé que la clé d'Alice était compromise. La commande -revoke l'a marquée comme révoquée (numéro de série 1001, raison Key Compromise) dans le carnet de bord de l'intermédiaire, puis j'ai généré les listes de révocation (CRL) de l'intermédiaire et de la racine. La vérification avec -crl_check_all donne error 23 : certificate revoked, alors que le certificat est encore dans sa période de validité. La commande -revoke écrit seulement dans le carnet privé de l'AC. Il faut ensuite générer la CRL (-gencrl), un document signé et publié : sans elle, les vérificateurs ne sauraient pas que le certificat est révoqué. Ma CRL est signée par Inter TP et valable 30 jours (Last Update 25 octobre 2026, Next Update 24 novembre 2026).

Automatisation (étape 7)

Le script pki.sh regroupe les étapes 2 à 4 et utilise set -euo pipefail : il s'arrête à la première erreur. Il refuse aussi de s'exécuter si les dossiers racine/ et inter/ existent déjà, pour ne rien écraser. Testé dans un dossier vide, il construit toute la PKI et les trois vérifications finales donnent OK.

3. Questions de réflexion

Question 1 - Pourquoi la clé de la racine reste-t-elle hors ligne ? Que faire si l'intermédiaire est compromis ? Et la racine ?

La clé de la racine est le secret le plus précieux : celui qui la possède peut créer des intermédiaires et des certificats acceptés par tous ceux qui lui font confiance. Hors ligne, c'est-à-dire sur une machine non connectée au réseau et rangée en lieu sûr, elle est beaucoup moins exposée. Si l'intermédiaire est compromis, la racine le révoque (son numéro de série est ajouté à la CRL de la racine, qu'on republie). On crée ensuite un nouvel intermédiaire avec une nouvelle clé, et on réémet les certificats. Si la racine est compromise, on ne peut pas la révoquer, puisqu'elle est auto-signée : il faut créer une nouvelle racine, la redistribuer comme nouveau point d'ancrage à tous les clients, retirer l'ancienne et réémettre tous les certificats. C'est très coûteux, d'où la protection hors ligne.

Question 2 - À quoi sert pathlen:0 ? Quel risque si on l'oublie ?

pathlen:0 signifie que l'intermédiaire peut signer seulement des certificats de fin de chaîne (serveur, client), pas d'autres AC. Si on l'oublie et que l'intermédiaire est compromis, l'attaquant peut créer ses propres AC en dessous, et construire toute une hiérarchie cachée très difficile à contrôler. pathlen:0 limite donc les dégâts.

Question 3 - Pourquoi la clé privée du serveur est-elle générée par le serveur et non par l'AC ?

Une clé privée ne doit jamais quitter son propriétaire. Si l'AC la générait, elle la connaîtrait et pourrait se faire passer pour le serveur, et la clé devrait en plus voyager sur le réseau, ce qui l'exposerait. Avec une demande de certificat (CSR), le serveur envoie seulement sa clé publique et une preuve qu'il possède la clé privée correspondante (la CSR est signée avec elle).

Question 4 - Le CN et le SAN contiennent www.tp.local : lequel les clients modernes vérifient-ils ?

Ils vérifient le SAN (noms alternatifs). Le nom commun (CN) est ancien et n'est plus utilisé pour vérifier un nom de domaine quand un SAN est présent. C'est pourquoi le certificat du serveur contient www.tp.local et tp.local dans le SAN.

Question 5 - Quelles sont les limites des CRL ? Comparaison avec OCSP et les certificats très courts.

Une CRL n'est mise à jour que périodiquement (ici tous les 30 jours au plus) : entre la révocation et la prochaine publication, un certificat révoqué peut être accepté. Elle peut aussi devenir longue, et le client doit la télécharger. OCSP interroge un serveur sur l'état d'un seul certificat, avec une réponse plus récente et plus légère, mais il dépend d'un serveur disponible et révèle à ce serveur les sites consultés (limite réduite par l'agrafage OCSP). Des certificats de très courte durée rendent la révocation presque inutile, car un certificat volé expire vite, mais ils imposent de renouveler automatiquement les certificats.

Question 6 - Politique de certification minimale (dix lignes)

- Qui peut demander : seuls les administrateurs du laboratoire pour les serveurs, et les utilisateurs enregistrés pour leur certificat client.
- Chaque demandeur génère sa propre clé et envoie une CSR ; la clé privée ne quitte jamais sa machine.
- Identité vérifiée : pour un serveur, on contrôle que le demandeur gère le nom demandé ; pour un client, on vérifie l'identité de la personne avant l'émission.
- Les noms sont limités à tp.local (contraintes de nom de l'intermédiaire).
- Durées : racine 10 ans, intermédiaire 5 ans, serveurs et clients 90 jours.
- Clés : P-384 pour la racine (hors ligne), P-256 pour l'intermédiaire et les certificats ; fichiers de clés en lecture réservée à leur propriétaire.
- Chaque certificat de fin de chaîne a CA:FALSE et un seul usage : serveur TLS ou client TLS.
- Révocation : sur demande du titulaire ou en cas de suspicion de compromission, avec le motif enregistré.
- Publication : la CRL est régénérée après chaque révocation et au moins tous les 30 jours.
- Si l'intermédiaire est compromis, la racine le révoque et on crée un nouvel intermédiaire.